



-GLOBAL EXPLORER-

-INFORMATION TECHNOLOGY PRACTICAL ASSESSMENT
TASK-GRADE 12

-REUBEN WALKER-



DOCUMENT INDEX

Contents

DOCUMENT INDEX.....	1
INTRODUCTION (TASK-1A).....	3
Scenario Chosen:	3
Solution Program:	3
Problem Description:	3
Solution Implementation:	3
USER REQUIREMENTS (TASK-1B)	4
DATABASE DESIGN (TASK-2)	5
TblNorthAmerica	5
TblSouthAmerica	5
TblAfrica	5
TblEurope	6
TblRussiaAsia.....	6
TblAustralia.....	6
TbFlightRecords	7
TblUser	7
Diagramatic Representation of Relational Structure	8
DATA STRUCTURES (TASK-3A).....	9
Unit Landing_Frm	9
Unit Login_Frm	9
Unit Receipt_Frm.....	9
Unit SignIn _Frm.....	10
Unit InitialSelect _Frm.....	10
Unit Security_Frm	10
Unit DestinationChoose _Frm.....	11
Unit Bookings_Frm	11
Unit AccountEdit_Frm	12
Unit Admin_Frm	12
Unit Exp _U	13
Unified Modelling Language Diagram.....	14

ARRAYS (TASK-3B)	15
TEXTFILES (TASK-3C)	15
PROGRAM FLOW (TASK-4A).....	17
USER INTERFACE DESIGN (TASK-4B).....	18
Sign-Up Window (Inclusive of component examples).....	18
Administrative Interface Window (Inclusive of component examples)	19
INPUT-PROCESSING-OUTPUT DIAGRAMS (TASK-5)	23
SignIn_Frm	27
Booking_Frm	28

INTRODUCTION (TASK-1A)

Scenario Chosen:

Route/Trip planning: A system that can be used to:

- Plan/Compile the best/most economical route from the point of departure to the required destination. The system could be used to assist employees of delivery services, tourists and other travellers in the public and private sectors.
- Plan routes/trips which include different modes of transportation to travel from one place to another

Solution Program:

Introducing GlobalExplorer, a powerful software application written in Object Pascal code, constructed in Delphi 2010 IDE, designed to help South African travellers effortlessly organize flights to every inter-continental travel, be it for business, holiday or customer desire. GlobalExplorer is tailored to streamline the planning, booking and recording process, offering users a seamless experience when booking international trips.

Problem Description:

The most difficult problem to overcome when organising excursions or holiday trips are the organisational and logistical challenges regarding the booking, and planning of the flight. GlobalExplorer will solve such problems by making trip/route planning simple affordable and user friendly.

Another mentionable design challenge would be the efficient cataloguing and saving of user data, flight records and location information. GlobalExplorer will solve such problems by implementing scalable and streamlined databasing technologies to archive, retrieve and generate important information for users and for the GlobalExplorer administrative team.

Solution Implementation:

Upon entering the app, users are greeted with a user-friendly interface that allows them to easily save personal details, input their desired destination, travel dates, and budget. The software will provide the user with an extensive list of options to decide from, all while showcasing the best deals and special offers tailored to the user's search.

What sets GlobalExplorer apart is its exclusive focus on outbound flights from South Africa. Whether traveling to Europe, Asia, North America, South America, or Africa the application curates a wide range of options from local airlines and international carriers operating out of major South African airports such as Port Elizabeth International, King Shaka Durban International, Or Tambo Johannesburg International, Cape Town International or Bram Fischer International. Another amazing benefit of choosing GlobalExplorer's flight planning system is its extensive Administrative Interface providing easy, understandable, navigable and most of all full access to GlobalExplorer's servers.

USER REQUIREMENTS (TASK-1B)

System User	Activity	Acceptance Test	Limitations
➤ General User Test Login: E-Mail: TestEntry@website.com Password: @TestAccount101	➤ “I want to be able to navigate easily through the app” ➤ I want to be able to plan flights, and have all details specified to me like time and costs	➤ Users will be presented with a simplistic and navigational user interface making decisions expedient ➤ Location, costs and times will be calculated and displayed to the user	➤ Choices can only vary to the locations provided ➤ All costs will be displayed in ZAR currency, Values displayed will be in metric units
➤ Application administrator Login: E-Mail: #Admin@GlobalExplorer Password: @dmin_#2000	➤ “I want to be able to have full control over database values and locations so I can adjust costs and prices for inflation”	➤ The application administrator will be presented with a decisive and applicable interface where they can extract, update, enter and delete records	➤ Only the database structure will limit the administrator

DATABASE DESIGN (TASK-2)

TblNorthAmerica

PK	FIELD	DATATYPE	FIELD SIZE
	AirPortID	Short Text	25
	AirportName	Short Text	25
	State	Short Text	25
	Address	Short Text	55
	WorkingHours	Short Text	25
	FlightCost	Number	Currency
	Rating	Short Text	25

TblSouthAmerica

PK	FIELD	DATATYPE	FIELD SIZE
	AirPortID	Short Text	25
	AirportName	Short Text	25
	State	Short Text	25
	Address	Short Text	55
	WorkingHours	Short Text	25
	FlightCost	Number	Currency
	Rating	Short Text	25

TblAfrica

PK	FIELD	DATATYPE	FIELD SIZE
	AirPortID	Short Text	25
	AirportName	Short Text	25
	State	Short Text	25
	Address	Short Text	55
	WorkingHours	Short Text	25
	FlightCost	Number	Currency
	Rating	Short Text	25

TblEurope

PK	FIELD	DATATYPE	FIELD SIZE
	AirPortID	Short Text	25
	AirportName	Short Text	25
	State	Short Text	25
	Address	Short Text	55
	WorkingHours	Short Text	25
	FlightCost	Number	Currency
	Rating	Short Text	25

TblRussiaAsia

PK	FIELD	DATATYPE	FIELD SIZE
	AirPortID	Short Text	25
	AirportName	Short Text	25
	State	Short Text	25
	Address	Short Text	55
	WorkingHours	Short Text	25
	FlightCost	Number	Currency
	Rating	Short Text	25

TblAustralia

PK	FIELD	DATATYPE	FIELD SIZE
	AirPortID	Short Text	25
	AirportName	Short Text	25
	State	Short Text	25
	Address	Short Text	55
	WorkingHours	Short Text	25
	FlightCost	Number	Currency
	Rating	Short Text	25

- **AirPortID**- Used to identify each Airport individually, First Integer place represents continent values
 - “1xxxx”- North America
 - “2xxxx”- South America
 - “3xxxx”- Africa
 - “4xxxx”- Europe
 - “5xxxx”- Asia
 - “6xxxx”- Australia
- **AirportName**- Name of Airport
- **State**- State in which airport is situated
- **Address**- Airport's Address
- **WorkingHours**- Open Hours of Airport
- **FlightCost**- How much a flight from that airport is valued at
- **Rating**- 5 star average rating of overall passenger experience

TbFlightRecords

PK	FIELD	DATATYPE	FIELD SIZE
	FlightID	Short Text	5
FK	PassengerID	Short Text	14
	Initial Airport	Short Text	25
FK	AirportID	Short Text	25
	DestinationContinent	Short Text	25
	NoSeats	Number	Integer
	NetProfit	Number	Long Integer

- **FlightID**- Used to identify each Flight Record individually
- **PassengerID**- Foreign Key
- **Initial Airport**- Airport from which the passenger booked
- **AirportID**- Foreign Key, The airport to which they will be flying.
- **DestinationContinent**- The Continent to which they will be travelling
- **NoSeats**- Total number of seats booked
- **NetProfit**- Net profit generated

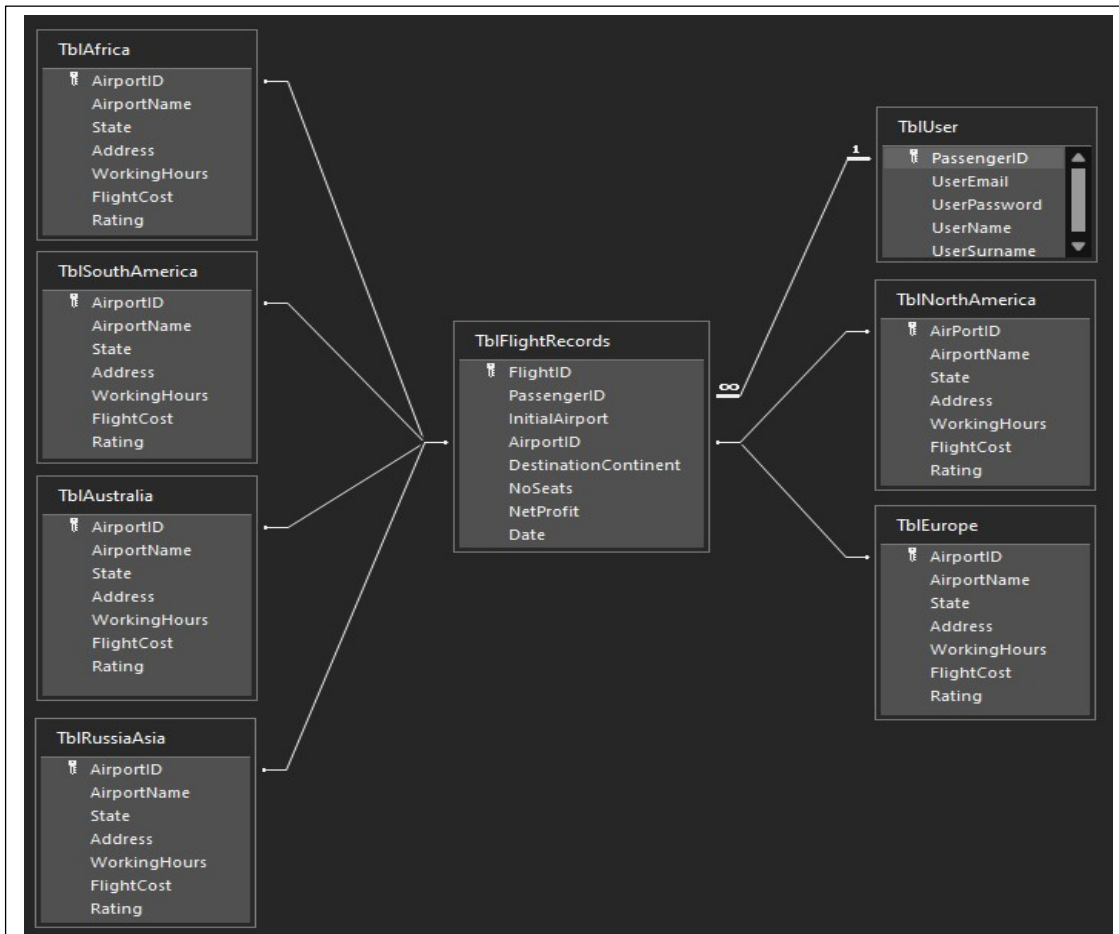
All Fields are used for statistical analysis and record keeping, facilitating the tracking of company expense and income while simultaneously monitoring trends and emergent patterns

TblUser

PK	FIELD	DATATYPE	FIELD SIZE
	PassengerID	Short Text	14
	UserEmail	Short Text	55
	UserPassword	Short Text	55
	UserName	Short Text	55
	UserSurname	Short Text	55

- **PassengerID**- Used to identify each passenger/user individually, it is their ID number issued by home affairs.
- **UserEmail**- Users Contact Email, used as login details
- **UserPassword**- The Complex string to authenticate and verify user access
- **UserName**- The users first name, Displayed on welcome page
- **UserSurname**- The users surname

Diagrammatic Representation of Relational Structure



The structure of Global Explorers Database (Centralized Table: "TblFlightRecords" receives the "many" side of all relationships)

DATA STRUCTURES (TASK-3A)

- All On ComponentLeaveEnter and On ComponentMouseLeave procedures have been omitted as they serve only to load assets for animations.
- All Procedures/Functions are set as public unless otherwise specified.

Unit Landing_Frm

TYPE	NAME	PROCESSING
procedure	FormActivate(Sender: TObject);	Runs Animations and Component Setup
procedure	TimerBDTimer(Sender: TObject);	Times Animations
procedure	ImgExitClick(Sender: TObject);	Closes Application
procedure	ImgLoginClick(Sender: TObject);	Proceeds to Login Form
procedure	ImgSignupClick(Sender: TObject);	Proceeds to Sign Up Form
Private	ITimeCount: integer;	Stores the timing variable

Unit Login_Frm

TYPE	NAME	PROCESSING
procedure	AdminCheck;	Runs a scan through a text-file serving as the admin key, if correct details sets admin status to true;
procedure	ImgExitClick(Sender: TObject);	Closes Application
procedure	ImgReturnClick(Sender: TObject);	Returns to Landing Form
procedure	FormActivate(Sender: TObject);	Runs Animations and Component Setup
procedure	TimerBDTimer(Sender: TObject);	Times Animations
procedure	ImgConfirmClick(Sender: TObject);	If Admin status true, proceeds to admin form Runs Scan Through Database, if correct user is detected, proceeds to Initial Select form
private	ltimeCount: integer;	Stores the timing variable
private	IsAdmin: boolean;	Defines admin status
public	Spassword:string;	Stores user password
public	SEmail:string;	Stores user Email address
public	SID:string;	Stores user Identification number
public	SUserName:string;	Stores User name
public	SuserSurname:string;	Stores User Surname

Unit Receipt_Frm

TYPE	NAME	PROCESSING
procedure	FormActivate(Sender: TObject);	Runs Animations and Component Setup
procedure	TimerBDTimer(Sender: TObject);	Times Animations, executes receipt generation
procedure	ImgFinishClick(Sender: TObject);	Executes print if selected,, proceeds to landing form
procedure	ImgReturnClick(Sender: TObject);	Returns to Booking Form
procedure	ImgAccEditClick(Sender: TObject);	Proceeds to security form
private	ltimeCount:integer;	Stores the timing variable

Unit SignIn_Frm

TYPE	NAME	PROCESSING
procedure	FormActivate(Sender: TObject);	Runs Animations and Component Setup
procedure	TimerBDTimer(Sender: TObject);	Times Animations
procedure	ImgExitClick(Sender: TObject);	Closes Application
procedure	ImgReturnClick(Sender: TObject);	Returns to Landing Form
procedure	Validation;	Validates All entered information, so it can be uploaded to Database
procedure	ImgConfirmClick(Sender: TObject);	Runs Validation, Uploads Information to Database, proceeds to login form
private	IsOkay: boolean;	Stores the validation status of information entered
private	ltimeCount: integer;	Stores the timing variable

Unit InitialSelect_Frm

TYPE	NAME	PROCESSING
procedure	FormActivate(Sender: TObject);	Runs Animations and Component Setup
procedure	TimerBDTimer(Sender: TObject);	Times Animations
procedure	ImgExitClick(Sender: TObject);	Closes Application
procedure	ImgReturnClick(Sender: TObject);	Returns to Login Form
procedure	ImgCPTClick(Sender: TObject);	Sets Initial Airport, proceeds to destination Continent Choose Form
procedure	ImgPEClick(Sender: TObject);	Sets Initial Airport, proceeds to destination Continent Choose Form
procedure	ImgKNGClick(Sender: TObject);	Sets Initial Airport, proceeds to destination Continent Choose Form
procedure	ImgBFNClick(Sender: TObject);	Sets Initial Airport, proceeds to destination Continent Choose Form
procedure	ImgJHBClick(Sender: TObject);	Sets Initial Airport, proceeds to destination Continent Choose Form
private	lTimeCount: integer;	Stores the timing variable
public	Airport:string;	Stores initial airport name

Unit Security_Frm

TYPE	NAME	PROCESSING
procedure	ImgExitDbClick(Sender: TObject);	Closes application
procedure	ImgReturnClick(Sender: TObject);	Returns to booking Form
procedure	ImgConfirmClick(Sender: TObject);	Checks if your password is correct, proceeds to Account Edit Form
private	spass:string;	Stores users password being entered

Unit DestinationChoose_Frm

TYPE	NAME	PROCESSING
procedure	FormActivate(Sender: TObject);	Runs Animations and Component Setup
procedure	TimerBDTimer(Sender: TObject);	Times Animations
procedure	ImgExitClick(Sender: TObject);	Closes Application
procedure	ImgReturnClick(Sender: TObject);	Returns to Initial Select Form
procedure	procedure TimerSelectTimer(Sender: TObject);	Times Animations
procedure	ImgConfirmClick(Sender: TObject);	Sets Destination Continent, proceeds to bookings form
procedure	ImgRightClick(Sender: TObject);	Moves Pointer Right
procedure	ImgLeftClick(Sender: TObject);	Moves Pointer Left
private	lTimeCount	Stores the timing variable
private	ltimeFormBDCount	Stores the timing variable for continent change animations
public	lselect:Integer;	Stores continent choice number

Unit Bookings_Frm

TYPE	NAME	PROCESSING
procedure	FormActivate(Sender: TObject);	Runs Animations and Component Setup
procedure	TimerBDTimer(Sender: TObject);	Times Animations
procedure	ImgExitClick(Sender: TObject);	Closes Application
procedure	ImgReturnClick(Sender: TObject);	Returns to Destination Continent Choose Form
procedure	procedure SpnFCChange(Sender: TObject);	Records number of first-class seats and baggage
procedure	SpnLCChange(Sender: TObject);	Records number of Lower-class seats and baggage
procedure	SpnMCChange(Sender: TObject);	Records number of middle-class seats and baggage
procedure	DBSearchCellClick(Column: TColumn);	Displays 5 star rating
procedure	EnterSQL(SQLString: String);	Runs Specified SQL
procedure	ImgSearchClick(Sender: TObject);	Executes EnterSQL, searches for entered information and displays
private	Rating:real;	Stores 5 star rating from database
private	Srating :string;	Stores string rating from database
private	SSearch :string;	Stores search text for database
private	Scon :string;	Stores continent name
private	lSeats:integer	Stores number of seats being booked
private	lbaggage:integer	Stores value of baggage weight in Kilograms
private	ltimeCount:integer	Stores the timing variable

Unit AccountEdit_Frm

TYPE	NAME	PROCESSING
procedure	ImgExitDbClick(Sender: TObject);	Closes application
procedure	ImgReturnClick(Sender: TObject);	Returns to booking Form
procedure	FormActivate(Sender: TObject);	Runs Animations and Component Setup
procedure	TimerBDTimer(Sender: TObject);	Times Animations, executes receipt generation
procedure	ImgEditNameClick(Sender: TObject);	Validates, Uploads new Information in database
procedure	ImgEditSurnamClick(Sender: TObject);	Validates, Uploads new Information in database
procedure	ImgEditEmailClick(Sender: TObject);	Validates, Uploads new Information in database
procedure	ImgEditPassClick(Sender: TObject);	Validates, Uploads new Information in database
procedure	procedure ImgEditIDClick(Sender: TObject);	Validates, Uploads new Information in database
procedure	Edit(FieldNo: Integer; New, Old: string);	Runs SQL to update values
procedure	Validation;	Validates all entered information
procedure	fetch;	Retrieves current user values
private	ltimeCount:integer;	Stores the timing variable
private	IsOkay:Boolean;	Defines validation status

Unit Admin_Frm

TYPE	NAME	PROCESSING
procedure	FormActivate(Sender: TObject);	Runs Animations and Component Setup
procedure	timerBDTimer(Sender: TObject);	Times Animations
procedure	GenDetails(SQL:string);	Sequel database code executor to retrieve information for inserting and editing records
procedure	ExecuteDB(SQL: string);	Sequel database code executor
procedure	EditDB(SQL: string);	Sequel database code executor
procedure	UserIDCheck(ID:string);	Checks the validity and existence of users Identification number
procedure	CheckEmpty;	Validates inputs for Airport tables
procedure	CheckEmpty2;	Validates inputs for Flight Record table
procedure	CheckEmpty3;	Validates inputs for User tables
procedure	IDValid;	Checks Validity and existence of Airport Identification Numbers
procedure	CheckTime;	Check designed specifically to validate working hours input format
procedure	FlightIDVal;	Checks Validity and existence of flight record Identification numbers

Unit Exp _U

TYPE	NAME	PROCESSING
constructor	create(ConCont, ConInitial, ConID, ConName, ConSurname, ConEmail: string; ConNoLC, ConNoMC, ConNoFC, ConOnBoardBag, ConBag: integer; ConAirID, ConAirName, ConState, ConAddress, ConWH: string; ConCostPer: real);	Creates and populates ObjExp
procedure	CalcCost;	Calculates net costs for user
procedure	RecordFlight;	Updates Values In Database Flight Records
procedure	PrintPDF;	Prints a receipt in PDF format for user
function	PrintReceipt: string	Result is receipt string
function	getFinitialAirport: string;	Result is initial Airport Name
function	getPassengerID: string;	Result is passenger ID
function	getpassengerName: string;	Result is Passenger Name
function	getpassengerSurname: string;	Result is passenger Surname
function	getpassengerEmail: string;	Result is passenger Email
function	getCostPer: real;	Result is Cost of each seat in said flight
function	getirportID: string;	Result is Airport ID
function	getairportName: string;	Result is Airport name
function	getState: string;	Result is Airport Sate
function	getworkingHours: string;	Result is Airport's working Hours
private	FInitialAirport: string;	Stores initial airports name
private	FPassengerID: string;	Stores passenger ID
private	FpassengerName: string;	Stores passenger name
private	FpassengerSurname: string;	Stores passenger surname
private	FpassengerEmail: string;	Stores passenger Email
private	FdestinationCon: string;	Stores destination continent name
private	Fcost: real;	Stores cost value per seat from database
private	FairportID: string;	Stores airport ID
private	FairportName: String;	Stores Airport name
private	FState: string;	Stores State-name of airport
private	Faddress: string;	Stores address of airport
private	FworkingHours: string;	Stores working hours of airport
private	FOnBoardBag: integer;	Stores weight of net on board baggage
private	FBag: integer;	Stores weight of bags
private	Rtotal: real;	Stores net costs for baggage and flight
private	RBagCost: real;	Stores baggage cost of all bags
private	Seats: integer;	Stores the number of seats

Unified Modelling Language Diagram

TExp

- FInitialAirport: string;
- FpassengerName: string;
- FpassengerSurname: string;
- FpassengerEmail: string;
- FdestinationCon: string;
- Fcost: real;
- FairportID: string;
- FairportName: String;
- FState: string;
- Faddress: string;
- FworkingHours: string;
- FOnBoardBag: integer;
- FBag: integer;
- Rtotal: real;
- RBagCost: real;
- FSeats: real;
- ArrFinances: Array [1..2 , 1..3] of real;

- + constructor create(ConCont, ConInitial, ConID, ConName, ConSurname, ConEmail: string; ConNoLC, ConNoMC, ConNoFC, ConOnBoardBag, ConBag: integer; ConAirID, ConAirName, ConState, ConAddress, ConWH: string; ConCostPer: real);
- + procedure CalcCost;
- + procedure RecordFlight;
- + procedure PrintPDF;
- + function PrintReceipt: string;
- + function getFInitialAirport: string;
- + function getPassengerID: string;
- + function getpassengerName: string;
- + function getpassengerSurname: string;
- + function getpassengerEmail: string;
- + function getCostPer: real;
- + function getirportID: string;
- + function getairportName: string;
- + function getState: string;
- + function getworkingHours: string;

ARRAYS (TASK-3B)

ArrFinances: Array[1..2, 1..3] of Real;

- 2 rows: indexed from 1 → 2
- 3 columns: indexed from 1 → 3
- Each element of the array is of type real

ArrFinances Population Example			
-	Upper Class	Middle Class	Lower Class
No Of Seats	2	3	7
Cost Per Seat	Original Cost * 2.2	Original Cost	Original Cost * 0.5

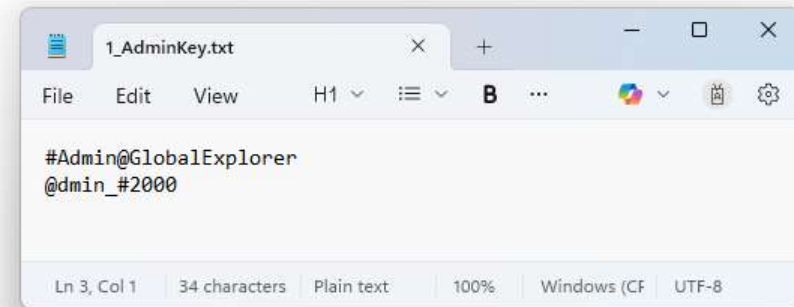
The two-dimensional array (ArrFinances: Array[1..2, 1..3] of Real) is used to store financial data related to different seating classes: High, Middle and Lower. The first dimension (rows) has two parts: Row 1 holds the number of seats for each class, while Row 2 stores the cost per seat for each corresponding class. The second dimension (columns) distinguishes between the classes: Column 1 for Upper-Class, Column 2 for Middle-Class, and Column 3 for Lower-Class. For example, ArrFinances[1,2] represents the number of Middle-Class seats, and ArrFinances[2,3] represents the cost of a Lower-Class seat. This structure allows for efficient calculations, such as multiplying seat count by cost to determine total revenue per class.

TEXTFILES (TASK-3C)

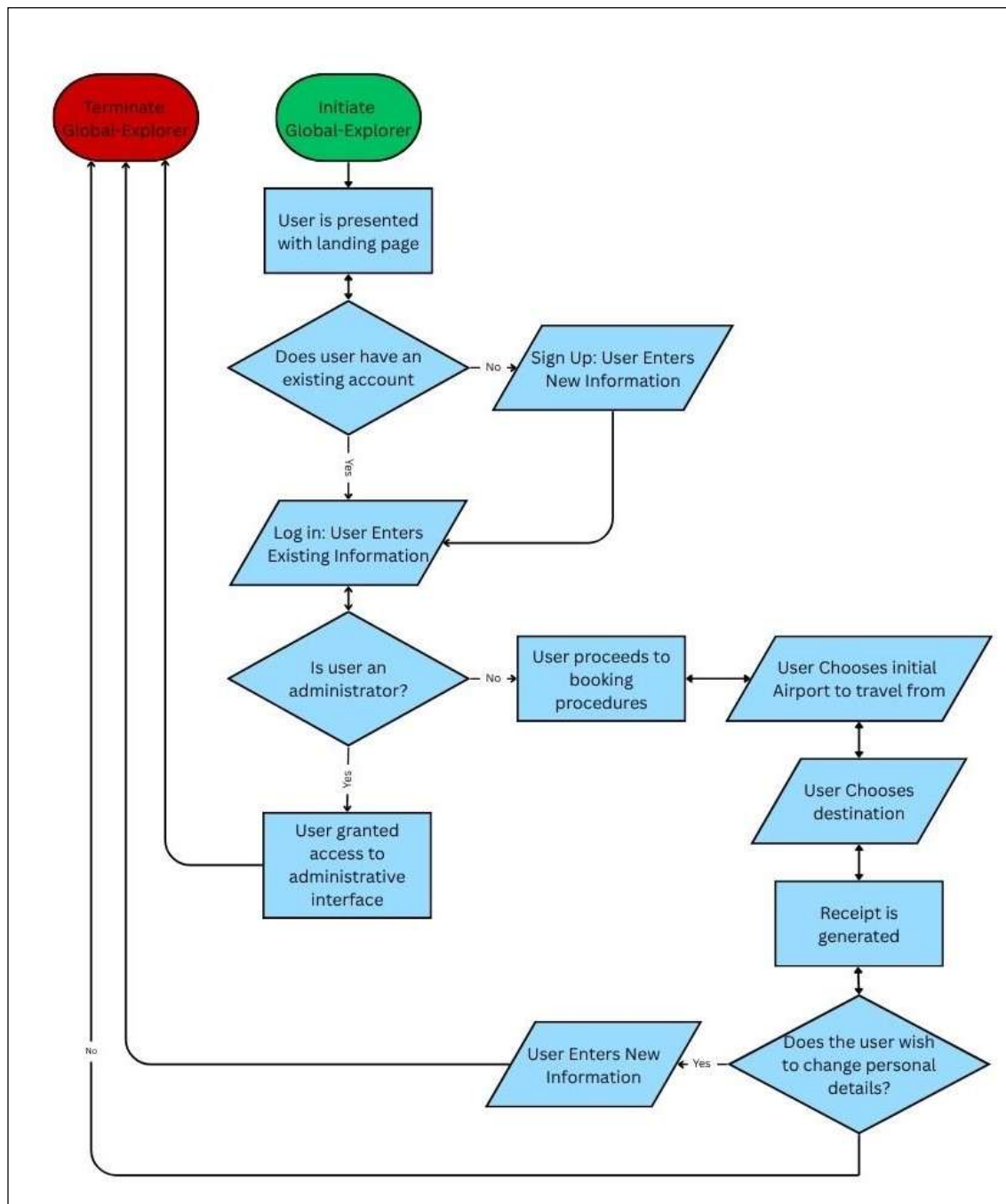
1_AdminKey.txt

- Declared as: AdminKey: textfile;

This text file serves as an external admin key for a program, designed to securely store administrator login credentials without embedding them directly into the application code. It contains two lines: the first representing the admin username **#Admin@GlobalExplorer** and the second the corresponding password **@dmin #2000**. By reading these values at runtime, the program can authenticate admin-level access by comparing them to user input. This method offers flexibility and security, allowing credentials to be updated without recompiling the software and enabling restricted access through file system permissions. In practical implementation, such as in Delphi, the program reads the file line by line using standard file I/O functions and temporarily stores the values for validation. This approach ensures that administrative access is modular, easy to maintain, and securely managed through external file-based verification.



PROGRAM FLOW (TASK-4A)



- All necessary validation such as presence checks, format checks, data-type checks, lengths checks and value checks are performed on user required inputs.

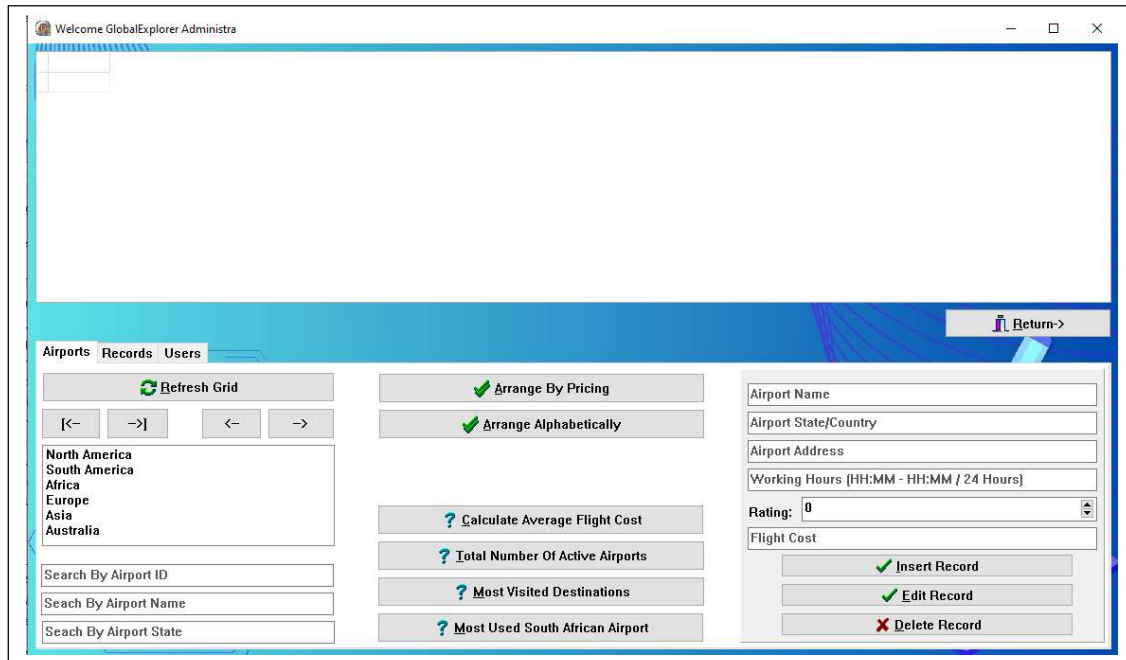
USER INTERFACE DESIGN (TASK-4B)

Sign-Up Window (Inclusive of component examples)



- TFrmSignIn = class(TForm)
- ImgBD: TImage;
- ImgExit: TImage;
- ImgReturn: TImage;
- TimerBD: TTimer;
- EdtID: TEdit;
- EdtName: TEdit;
- EdtSUrname: TEdit;
- EdtEmail: TEdit;
- ImgConfirm: TImage;
- EdtPass: TEdit;
- MemInfo: TMemo;

Administrative Interface Window (Inclusive of component examples)



- TFrmAdmin = class(TForm)
- ImgBD: TImage;
- DBGridAdmin: TDBGrid;
- PgCtrl: TPageControl;
- TabSheet1: TTabSheet;
- TabSheet2: TTabSheet;
- TabSheet3: TTabSheet;
- EdtSearchIDAir: TEdit;
- EdtSearchNameAir: TEdit;
- EdtSearchStateAir: TEdit;
- BtnFirstAir: TButton;
- BtnLastAir: TButton;
- BtnNextAir: TButton;
- BtnBackAir: TButton;
- BtnRefreshAir: TBitBtn;
- PnlInsert: TPanel;
- BitBtnEdit: TBitBtn;
- BitBtnInsert: TBitBtn;
- EdtAirName: TEdit;
- EdtAirState: TEdit;
- EdtAirAddress: TEdit;
- EdtWH: TEdit;
- EdtCost: TEdit;

- BitBtnSumRecords: TBitBtn;
- BitBtnDel: TBitBtn;
- SpnRating: TSpinEdit;
- LblRating: TLabel;

Welcome Global Explorer Administrator

Return →

Airports Records Users

Refresh Grid

[<-] [->] [->] [->]

Search By Record ID

Search By Passenger ID

Search By Email

Search By Passenger Name

User Details:

Enter Passenger's ID

Enter Initial Airport

Select Initial Airport

Enter Number Of Seats: 0

Enter Net Profit

Select Transaction Date: 2025/08/04

Destination Details:

Enter Destination Airport's ID

Insert Record

Edit Record

Delete Record

- bitBtnRefreshRec: TBitBtn;
- BtnFistRec: TButton;
- BtnLastRec: TButton;
- BtnbackRec: TButton;
- BtnNextRec: TButton;
- EdtRecordID: TEdit;
- EdtRecordName: TEdit;
- EdtEmailRec: TEdit;
- PnlRecord: TPanel;
- EdtNameRec: TEdit;
- BitBtnInsertRec: TBitBtn;
- BitBtnEditRec: TBitBtn;
- BitBtnDelRec: TBitBtn;
- EdtPassengerID: TEdit;
- EdtPassengerEmail: TEdit;
- EdtPassengerName: TEdit;
- EdtAddress: TEdit;
- EdtNetProfit: TEdit;
- EdtAirportID: TEdit;
- EdtAirportName: TEdit;
- EdtDestinationContinent: TEdit;

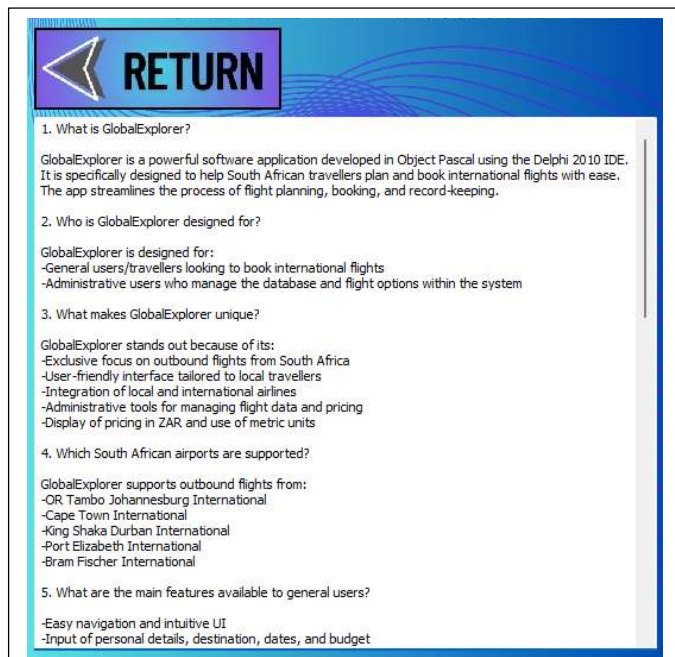
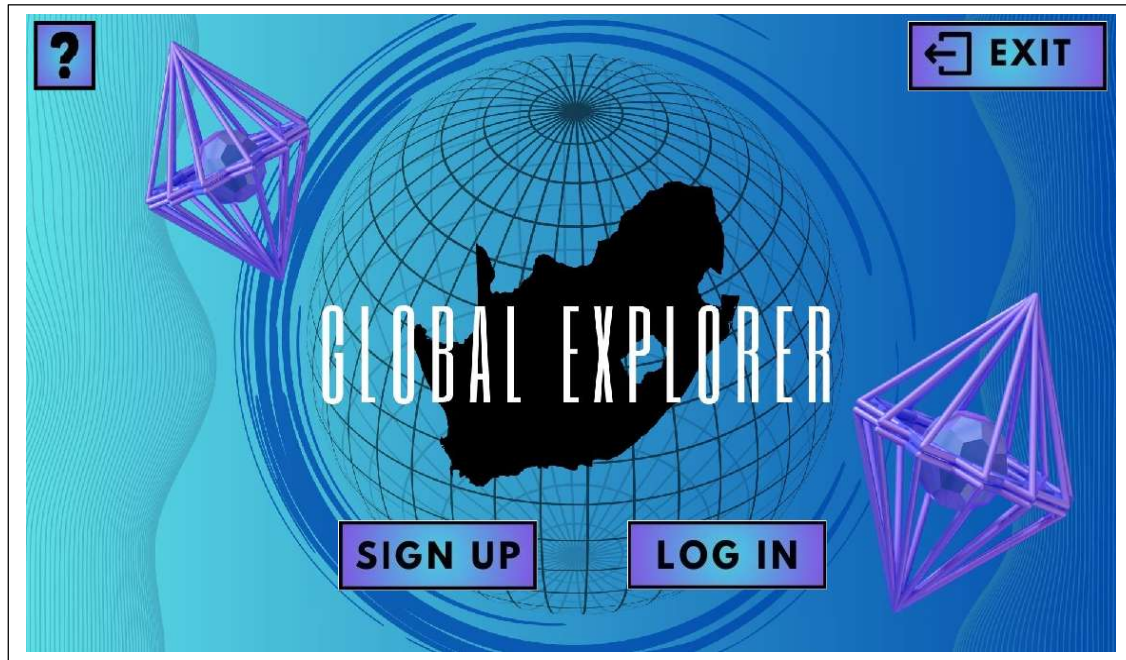
- EdtState: TEdit;
- BitBtnMostDes: TBitBtn;
- EdtPassengerSurname: TEdit;
- CmbInitialAirport: TComboBox;
- lblUser: TLabel;
- lblDestination: TLabel;
- SpnEdtNoSeats: TSpinEdit;
- lblSeats: TLabel;
- DateTrans: TDateTimePicker;
- lblDate: TLabel;
- BitBtnMostInitial: TBitBtn

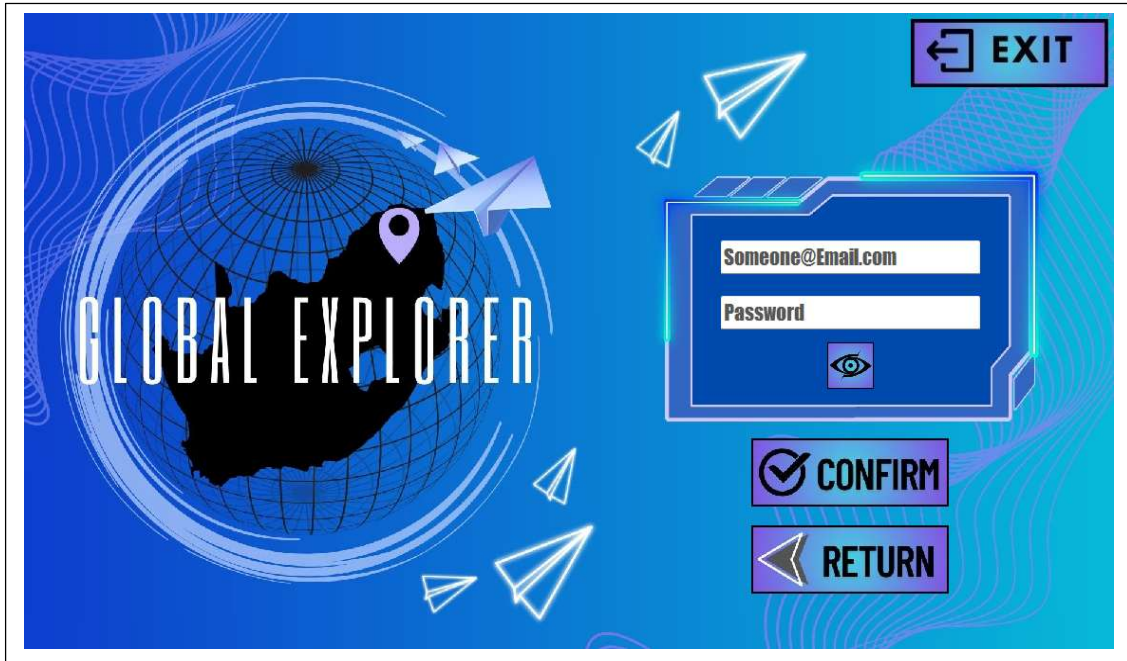
The screenshot displays the 'Global Explorer Administration' window with the 'Users' tab selected. The interface includes a 'Refresh Grid' button, navigation buttons, and search filters for User ID, Email, Name, and Surname. On the right, there are input fields for user details and three action buttons: 'Insert Record', 'Edit Record', and 'Delete Record'. A 'Return' button is located in the top right corner.

- EdtUserID: TEdit;
- EdtUserEmail: TEdit;
- EdtUserName: TEdit;
- EdtUserSurname: TEdit;
- PnlUserInsert: TPanel;
- EdtUserIDIns: TEdit;
- EdtUserNameIns: TEdit;
- EdtUserSurnameIns: TEdit;
- EdtPass: TEdit;
- EdtEmail: TEdit;
- BitBtnInsUser: TBitBtn;
- BitBtnEditUser: TBitBtn;
- BitBtnUserDel: TBitBtn;
- BitBtnHighPayCus: TBitBtn;
- BitBtnReturn: TBitBtn;

- ListBoxTblAir: TListBox;

User Interfaces:









THANKYOU FOR FLYING

→ Personal Details:

Client ID: 1000000000000

Client Name: User Test

→ Flying from:

Port Elizabeth International

→ Flying To:

Airport ID: 200001

Airport Name: Galeão International Airp

Airport Continent: South America

Airport State: Rio de Janeiro

Airport Address: Av. Vinte de Janeiro, s/n - Galeão

Departure Date: 2020/09/17

Flight Window: 24 Hours

→ Cost Details:

First Class Coverage: R38 550,60 For 1 Seats

Middle Class Coverage: R0,00 For 0 Seats

Lower Class Coverage: R0,00 For 0 Seats

On Board Baggage (5Kg Per Seat) Total: 5Kg

Total Cargo Baggage Fee For 1 is: R0,00

Net Coverage Cost: R38 550,60

 RETURN



FINISH 

☐ Print?

GLOBAL EXPLORER





EXIT

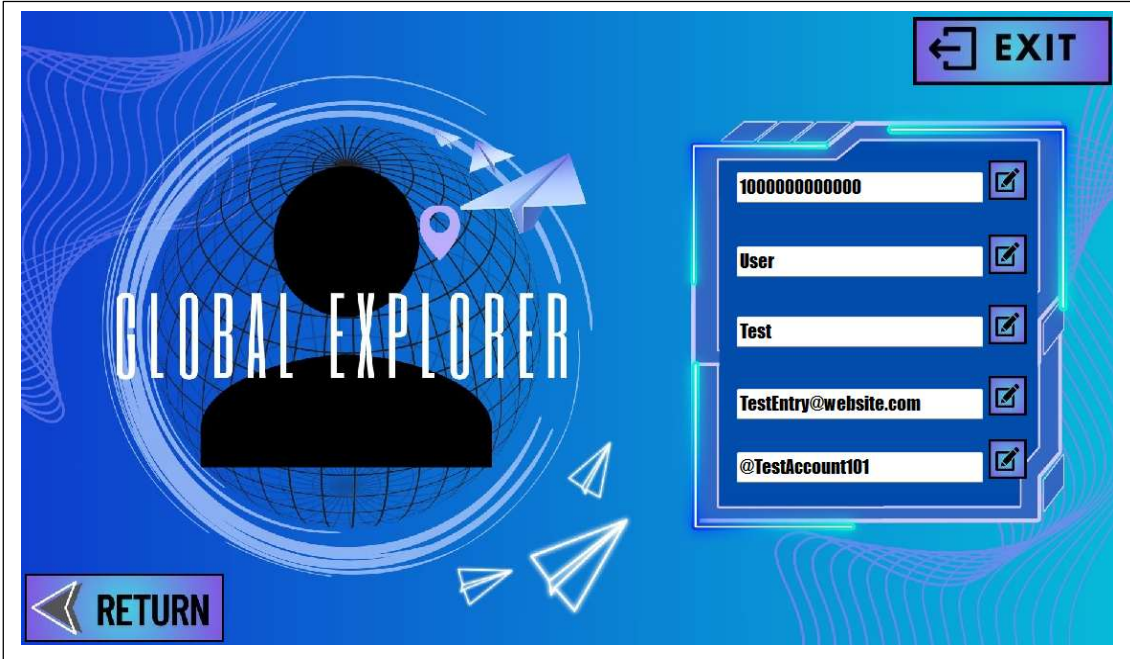




 CONFIRM

 RETURN





INPUT-PROCESSING-OUTPUT DIAGRAMS (TASK-5)

SignIn_Frm

INPUT	DATA TYPE & FORMAT	VALIDATION	PROCESSING	OUTPUT
EdtID: TEdit; (PassengerID)	String "xxxxxxxxxxxxx" 13 digit long ID	Length Check performed on input, if it is less than 13, passengers ID isn't accepted. Nul Check, if the variable is empty, passenger ID isn't accepted.	Passenger ID is uploaded to the database	Confirmation message
EdtName: TEdit; (UserName)	String "Name"	Nul Check, if the variable is empty, passengers' name isn't accepted.	Passengers name is uploaded to the database	Confirmation message
EdtSurname: TEdit; (USerSurname)	String "Surname"	Nul Check, if the variable is empty, passengers' surname isn't accepted.	Passengers surname is uploaded to the database	Confirmation message
EdtEmail: TEdit; (UserEmail)	String "text@website.com"	Nul Check, if the variable is empty, passengers' Email isn't accepted. Presence Check, if the presence of "@" is omitted, Email isn't accepted	Passengers Email is uploaded to the database	Confirmation message
EdtPass: TEdit; (UserPassword)	String "*****"	Nul Check, if the variable is empty, passengers' password isn't accepted. Presence check, if there is no capital letter nor unique symbol present, password isn't accepted. Length Check, if the variable is less than 5 charecters long, the password is rejected.	Passengers' password is uploaded to the database	Confirmation message

Booking_Frm

INPUT	DATA TYPE & FORMAT	VALIDATION	PROCESSING	OUTPUT
EdtSearch: TEdit; (Ssearch)	String "Search String"	Nul Check, if the variable is empty, User is prompted to enter a variable	Text is checked against the database and records are pulled from the database that that matches the search. Information is finally uploaded to the flight record database	All records are displayed on the database grid.
SpnMC: TSpinEdit; (NoMC) SpnLC: TSpinEdit; (NoLC) SpnFC: TSpinEdit; (NoFC)	Integer "xxx" Integer that contains number of seats booked in that class range	Maximum value of 50 is set and minimum 0	Cost per seat is retrieved from the database. The value is multiplied by the number of seats. The values, dependant on the type of class, lower class, middle class or first class, are processed with a multiplier, NoSeats * CostPerSeat * Multiplier Net total also summated. The anmount of baggage is summated, 5kg per booked seat. Information is finally uploaded to the flight record database	Costs are added to the receipt
OnDatabaseClick/ OnDatabaseChange	N/A This is a click based input that finalizes the upload of all inputs on the booking page	Player's choice is set to where the pointer is	Information is finally uploaded to the flight record database.	All Attributes are reordered from the database and added to the receipt
SpnPP: TSpinEdit; (BaggagePerPErson)	Integer "xx"	Maximum value of 70, minimum of 0	Amount multiplied by 300 per kg per person. Baggage cost is summated to net costs	Costs displayed on receipt